BrainAI

ProSuite Industrial

API & WRAPPER SUITE

PRODUKT-DOKUMENTATION

BrainAI (Inhaber: Sascha Alexander Köhne)

BrainAI Inhaber: Sascha Alexander Köhne	PROSUITE INDUSTRIAL	🟢 ÖFFENTLICHE KLASSIFIZIERUNG
Core Architecture & Development	**ProHash Integrations-Handbuch**	Zugriffsstufe: Öffentlich / Beta

---

Revisionshistorie / Revision History

Version	Datum	Autor	Status	Beschreibung
1.0.0	01.01.2025	S. Köhne	Stable	Initial Release of Wrapper Documentation
2.0.0	27.02.2026	S. Köhne	Stable	Full API Integration & Streaming Guide

1 Produktübersicht

Die **ProHash Wrapper-Suite** dient als Hochsprachen-Schnittstelle zur nativen ProHash-256 Engine („The Shredder“). Sie ermöglicht es Entwicklern, die Sponge-Konstruktion des Algorithmus effizient zu nutzen, ohne sich um manuelles Speichermanagement oder Bit-Manipulationen auf C-Ebene kümmern zu müssen.

Unterstützte Kernfunktionen:

- **Deterministisches Hashing:** Identische Ergebnisse über alle Plattformen (x86, x64, ARM).
- **Sponge-Streaming:** Verarbeiten von Datenmengen, die den verfügbaren RAM überschreiten, durch sequenzielle Updates.
- **Zero-Trust Memory Management:** Automatisches Nullen sensibler Zustandsdaten nach der Finalisierung.

2 Gemeinsame Grundlagen

Alle Wrapper bilden die native Struktur `ProHash_Ctx` (136 Bytes) ab, welche den 512-Bit Zustand (`belt`), den Byte-Zähler (`count`) und den Eingabepuffer verwaltet.

Standard-Workflow:

1. **Initialisierung:** Erzeugen des Kontexts und Setzen des Startzustands (`prohash_init`).
2. **Update:** Einspielen von Datenfragmenten (`prohash_update`).
3. **Finalisierung:** Padding und Generierung des 32-Byte Digests (`prohash_final`).

3 C# (.NET) Integration

Der C#-Wrapper nutzt `P/Invoke` und `unsafe` Blöcke, um den Kontext direkt im unmanaged Speicher zu verwalten.

- **Besonderheit:** Manuelle Speicherallokation via `Marshal.AllocHGlobal`, um GC-Verschiebungen zu verhindern.
- **Beispiel:**

```
using (var hasher = new ProHash()) {
    hasher.Update(Encoding.UTF8.GetBytes("Data"));
    byte[] result = hasher.Final();
    // result enthält 32 Bytes
}
```

4 Java (JNA) Integration

Die Java-Anbindung erfolgt über JNA mit striktem 8-Byte-Alignment für den `uint64_t` Zähler.

- **Maven/Gradle:** Erfordert `net.java.dev.jna`.
- **Beispiel:**

```
ProHash hasher = new ProHash();
hasher.update(fileContent);
byte[] digest = hasher.finalizeHash();
```

5 Python Integration

Nutzt `ctypes` und ist plattformagnostisch (lädt `.dll` oder `.so`).

- **Beispiel:**

```
hasher = ProHash()
hasher.update(b"Chunk 1")
```

```
digest = hasher.finalize() # Returns 32 bytes
```

6 JavaScript (Node.js) Integration

Verwendet `ffi-napi` für den nativen Aufruf und `Buffer` für effizienten Datentransfer.

- **Streaming-Support:** Ideal für `fs.createReadStream` Anbindungen.
- **Beispiel:**

```
const hasher = new ProHash();
hasher.update(Buffer.from("Input"));
const hash = hasher.finalize();
```

7 VB.NET Integration

Optimiert für OT-Systeme mittels `MarshalAs`-Attributen zur Sicherstellung fixer Array-Größen.

- **Beispiel:**

```
Dim hasher As New ProHash()
hasher.Update(dataBytes)
Dim result = hasher.FinalizeHash()
```

8 Best Practices & Sicherheit

- **Hex-Konvertierung:** Nutzen Sie für die Anzeige den Format-String `"x2"`, um den 32-Byte-Digest in einen 64-Zeichen Hex-String zu wandeln.
- **Threads:** ProHash-Kontexte sind nicht thread-sicher. Nutzen Sie pro Thread eine eigene Instanz.
- **Streaming:** Verwenden Sie bei Dateien > 100 MB immer die `Update`-Methode in Schleifen (Chunks), um den RAM-Verbrauch konstant niedrig zu halten.

© 2026 Sascha	✉️ Contact: koehne83@googlemail.com	🛡️ ProTU-Verified

Köhne		
CEO / System Architect	BrainAI Inhaber : Sascha Alexander Köhne	We don't need **BRUTEFORCE**, we know **PHYSICS**